The Mathematics of Backpropagation: Companion Derivations for `llm.mojo`

Evan Owen

***Abstract*—Working derivations of the backward pass for every operation in the GPT-2 forward pass, as implemented by the `llm.mojo` kernels. Sections are ordered by backward-derivation pedagogy—starting from the scalar loss and working toward the inputs—rather than by `llm.c` forward order, so that each gradient is derived only after the one it depends on. Each section anchors on the forward computation and leaves the derivation as structured workspace.**

CONTENTS

I. NOTATION AND CONVENTIONS

Shapes follow the kernel code, as collected in Table I.

Indices: b over batch, t over positions, i, j, k over channels or vocab entries. A row of activations at one position is $x_{bt} \in \mathbb{R}^C$.

a) The backward convention.: Every op receives the upstream gradient $\frac{\partial L}{\partial y}$ of the scalar loss L with respect to its output y, and must produce $\frac{\partial L}{\partial x}$ for each input x (and $\frac{\partial L}{\partial w}$ for each parameter) via the chain rule

$$\frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial x_i}. \tag{1}$$

Gradients into shared inputs *accumulate* (+=) because a tensor used by several consumers receives a contribution from each.

b) Reading order.: The forward pass runs encoder → layernorm → attention → matmul → GELU → softmax → cross-entropy. The sections below instead follow the order in which gradients are *derived*: starting at the loss (§III) and moving back toward the embeddings (§IX), each op reusing the upstream gradient established by the section before it. §X then reassembles these pieces into the end-to-end backward pass.

TODO: Decide and record: numerator vs. denominator layout, and where transposes land in the matmul gradients as a consequence.

II. WARM-UP: SCALAR CHAIN RULE ON A TINY NETWORK

A two-step composition to fix conventions before the tensor ops: $z = wx + b$, $a = \sigma(z)$, $L = \frac{1}{2}(a - y)^2$.

Derivation. Backward through the loss, then the sigmoid:

$$\frac{\partial L}{\partial a} = a - y \tag{2}$$
$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \tag{3}$$
$$\frac{\partial a}{\partial z} = \sigma(z) \cdot (1 - \sigma(z)) \tag{4}$$

and since $a = \sigma(z)$,

$$\frac{\partial L}{\partial z} = (a - y) \cdot \frac{\partial a}{\partial z} = (a - y) \cdot a \cdot (1 - a). \tag{5}$$

For the weight:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} \tag{6}$$
$$\frac{\partial z}{\partial w} = x \tag{7}$$
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = (a - y) \cdot a \cdot (1 - a) \cdot x. \tag{8}$$

For the bias:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b} \tag{9}$$
$$\frac{\partial z}{\partial b} = 1 \tag{10}$$
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a). \tag{11}$$

For the input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x} \tag{12}$$
$$\frac{\partial z}{\partial x} = w \tag{13}$$
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w = (a - y) \cdot a \cdot (1 - a) \cdot w. \tag{14}$$

TABLE I
SYMBOLS AND THEIR CORRESPONDING KERNEL PARAMETERS.

Symbol	Code	Meaning
B	`batch_size`	batch size
T	`seq_len`	sequence length
C	`channels`	model width (embedding dim)
V	`vocab_size`	real vocabulary size
V_p	`vocab_size_padded`	padded vocabulary (row stride)
H	`num_heads`	attention heads, head size $h = C/H$

Result. One chain back through the network, fanning out at the affine inputs:

$$\frac{\partial L}{\partial a} \;\to\; \frac{\partial L}{\partial z} \;\to\; \left\{\frac{\partial L}{\partial w},\ \frac{\partial L}{\partial b},\ \frac{\partial L}{\partial x}\right\} \tag{15}$$

with $\frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a)$ computed once and reused: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$.

III. Cross-Entropy Loss

Forward (`crossentropy_ohe_fwd`): given probabilities $p_{bt} \in \mathbb{R}^{V_p}$ and a target token $y_{bt} \in \{0, \dots, V-1\}$,

$$L_{bt} \;=\; -\log p_{bt,\,y_{bt}}. \tag{16}$$

The one-hot structure means only the target column participates.

A. Backward w.r.t. the probabilities

Derivation. **TODO:** Differentiate (16) w.r.t. $p_{bt,i}$; note which entries of $\frac{\partial L}{\partial p_{bt}}$ are nonzero and compare against the assign-only-the-target-column behavior of `crossentropy_ohe_bwd`.

Result. **TODO:** $\frac{\partial L}{\partial p_{bt,i}} = ?$

IV. Softmax

Forward: from logits $\ell_{bt} \in \mathbb{R}^{V_p}$ (real entries $i < V$),

$$p_{bt,i} \;=\; \frac{e^{\ell_{bt,i} - m_{bt}}}{\sum_{j<V} e^{\ell_{bt,j} - m_{bt}}}, \qquad m_{bt} = \max_{j<V} \ell_{bt,j}. \tag{17}$$

A. The Jacobian

Derivation. **TODO:** Derive $\frac{\partial p_i}{\partial \ell_j}$ for $i = j$ and $i \neq j$; show the max-subtraction does not change the derivative.

Result. **TODO:** $\frac{\partial p_i}{\partial \ell_j} = ?$ (express with δ_{ij}).

B. Fused softmax + cross-entropy

The composition is the case that matters for the model: backprop through (16) and (17) together and watch the Jacobian collapse.

Derivation. **TODO:** Chain the cross-entropy gradient through the softmax Jacobian and simplify to the famous one-liner. This is the math behind the planned fused kernel.

Result. **TODO:** $\frac{\partial L}{\partial \ell_{bt,i}} = ?$ (and note the padded region $i \geq V$).

V. Linear Layer (Matmul)

Forward: $Y = XW^\top + b$ with $X \in \mathbb{R}^{(B\cdot T)\times C_{\text{in}}}$, $W \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}$, $b \in \mathbb{R}^{C_{\text{out}}}$.

Derivation. **TODO:** Derive $\frac{\partial L}{\partial X}$, $\frac{\partial L}{\partial W}$, $\frac{\partial L}{\partial b}$ from (1); check every shape. Note where the reduction over $B \cdot T$ appears.

Result. **TODO:** $\frac{\partial L}{\partial X} = ?$, $\frac{\partial L}{\partial W} = ?$, $\frac{\partial L}{\partial b} = ?$

VI. GELU

Forward (tanh approximation, as in llm.c):

$$\mathrm{GELU}(x) \;=\; \tfrac{1}{2}\,x\left(1 + \tanh\!\left[\sqrt{2/\pi}\,(x + 0.044715\,x^3)\right]\right). \tag{18}$$

Derivation. **TODO:** Differentiate (18). Elementwise, so the chain rule is a pointwise product.

Result. **TODO:** $\mathrm{GELU}'(x) = ?$

VII. LayerNorm

Forward, per position $x \in \mathbb{R}^C$ with scale γ and shift β:

$$\begin{aligned} \mu &= \tfrac{1}{C}\sum_i x_i, \qquad \sigma^2 = \tfrac{1}{C}\sum_i (x_i - \mu)^2, \\ y_i &= \gamma_i\,\frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i. \end{aligned} \tag{19}$$

Derivation. **TODO:** The interesting one: μ and σ^2 both depend on every x_j, so $\frac{\partial y_i}{\partial x_j}$ has three terms. Work it via $\hat{x}_i = (x_i - \mu)/\sqrt{\sigma^2 + \varepsilon}$.

Result. **TODO:** $\frac{\partial L}{\partial x_j} = ?$, $\frac{\partial L}{\partial \gamma_i} = ?$, $\frac{\partial L}{\partial \beta_i} = ?$

VIII. Attention

Forward, per head: $A = \mathrm{softmax}\big(QK^\top/\sqrt{h} + M\big)$, $Y = AV$, with causal mask M and $Q = XW_Q$, etc.

Derivation. **TODO:** Backprop in stages: $\frac{\partial L}{\partial V}$ and $\frac{\partial L}{\partial A}$ from $Y = AV$; then $\frac{\partial L}{\partial (QK^\top)}$ through the row-wise softmax (reuse §IV); then $\frac{\partial L}{\partial Q}$, $\frac{\partial L}{\partial K}$. Track $\sqrt{h}$ and the mask.

Result. **TODO:** Gradient for each of Q, K, V.

IX. Embeddings (Encoder)

Forward: $x_{bt} = W_E[\,\mathrm{tok}_{bt}\,] + W_P[\,t\,]$.

Derivation. **TODO:** Gradients of a gather are a scatter-add; multiple (b, t) hitting the same token row accumulate. Connect to the $+=$ convention of §I.

Result. **TODO:** $\frac{\partial L}{\partial W_E} = ?$, $\frac{\partial L}{\partial W_P} = ?$

X. The Full Backward Pass

Compose §III–§IX in reverse forward order, including the residual stream (where gradients add) and weight tying between W_E and the LM head.

TODO: Write the end-to-end gradient flow for one transformer block, then the whole model.

Appendix A
Gradient Checking

Numerical check used to validate every derivation above:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + \epsilon e_i) - L(\theta - \epsilon e_i)}{2\epsilon}. \tag{20}$$

TODO: Record per-dtype tolerances and how they relate to `tests/_dtypes.py`.

Appendix B
Useful Identities

- $\frac{\mathrm{d}}{\mathrm{d}x} \log x = 1/x$; $\frac{\mathrm{d}}{\mathrm{d}x} e^x = e^x$; $\frac{\mathrm{d}}{\mathrm{d}x} \tanh x = 1 - \tanh^2 x$.
- log-sum-exp and its gradient: $\nabla \log \sum_j e^{x_j} = \mathrm{softmax}(x)$.
- **TODO:** Collect identities as they get used.